PROJECT EXPLANATION & PRESENTATION NOTES / SEPTEMBER 2026

EchoDrop.

A guide to the application, its code, and your demonstration.

EchoDrop is a city-themed message board built with Next.js, React, and MongoDB. Visitors can read conversations. Registered users can share short posts, comment, like posts, and manage their own posts.

Each post has a 24-hour active lifetime. The interface presents authors as "Someone in [city]" to keep the attention on the message.

> **Opening statement you can use**
> "EchoDrop is a full-stack message board for short conversations connected to a city. I use React for the interactive interface, Next.js for the pages and backend endpoints, and MongoDB for persistent storage. This presentation follows a message from the screen, through the server, into the database, and back."

What this project demonstrates

- A responsive interface with reusable components and clear interaction states.
- Authentication with hashed passwords and a signed cookie-based token.
- Database relationships between users, posts, comments, and votes.
- Server-side validation, ownership checks, and requests without application data caching.

Guide contents

01 / Project overview

02 / Architecture and folders

03 / Login and registration

04 / The feed

05 / Post details and comments

06 / Profile and About

07 / Shared interface and design

08 / MongoDB and models

09 / Authentication and access

10 / API reference

11 / Helpers and caching

12 / A request, explained step by step

13 / Presentation and demo plan

14 / Questions and current limits

Written against the current project files. Speaking notes describe implemented behavior; possible improvements are identified separately. No database credentials or private configuration values are included.

02 / THE BIG PICTURE

One project, three responsibilities

The frontend displays information and collects actions. The backend validates those actions and enforces access rules. MongoDB keeps the records after a page reload or application restart.

1. Browser	2. Next.js API	3. Mongoose	4. MongoDB
React pages and forms	Requests, validation, access	Schemas and queries	Persistent documents

A response travels back through the same layers as JSON, which the frontend uses to update the screen. The browser does not connect directly to MongoDB.

Technology	Role in EchoDrop
Next.js / App Router	File-based page routes and backend route handlers in one application.
React / JavaScript	Components, form interactions, local state, and rendering lists.
MongoDB / Mongoose	Document storage plus model definitions, validation, references, and queries.
bcryptjs / jsonwebtoken	Password hashing and signing/verifying authentication tokens.
CSS / lucide-react	Responsive styling and consistent interface icons.

Where to find the code

```
echodrop/
  app/           Pages, root layout, global CSS, and api/ routes
  components/    Brand, AppShell, AuthForm, and PostCard
  lib/           Database, authentication, requests, and query helpers
  models/        User, Post, Comment, and Vote schemas
  public/        Locally served city.jpg photograph
  scripts/       check-db.mjs connection check
  docs/          This presentation guide
  package.json   Dependencies and runnable commands
  next.config.mjs  API cache headers and developer indicator setting
```

`app/home/page.js` creates `/home`. `app/api/posts/route.js` creates `/api/posts`. A folder named `[id]` is a dynamic URL segment representing a particular post.

`jsconfig.json` maps `@/` to the project root, so imports such as `@/lib/api` avoid long relative paths. `package-lock.json` records resolved dependency versions; `node_modules/` contains installed packages and `.next/` contains generated Next.js output.

> **How to explain it**
>
> "Next.js contains both the interface and the API. Mongoose is the layer that lets the server work with MongoDB through defined models."

03 / ACCOUNT PAGES

Login and registration

Files: `app/page.js`, `app/register/page.js`, `components/auth-form.js`

The login page is the first screen at `/`. Registration is at `/register`. Both use the same `AuthForm` component. The `register` prop selects the fields, wording, and endpoint.

Element	What it does and why it exists
EchoDrop logo	Identifies the application and links to the login page on these screens.
Explore the feed	Opens the publicly readable feed without requiring registration.
City photograph	Connects the visual identity to the city theme. It is served from `public/city.jpg`.
Username	Used for login. New usernames require at least 3 characters; the form and model limit them to 50.
Email and city	Shown only during registration. Email identifies the account; city is attached to newly created posts.
Password and eye icon	Passwords are masked by default. The icon changes visibility without submitting the form. Registration requires at least 6 characters.
Primary button	Submits the form. Displays "Please wait..." and is disabled while a request is pending.
Error message	Displays a returned error, such as invalid credentials or a duplicate account.
Account link	Moves between login and registration without duplicating either form implementation.

What happens on submit

1. `event.preventDefault()` stops a normal browser form navigation.
2. `FormData` reads the named inputs, and `Object.fromEntries()` builds a plain object.
3. `api()` sends a JSON POST to the selected authentication endpoint.
4. The server either creates the account or checks the login details, then sets the authentication cookie.
5. On success, the page uses `router.replace("/home")` and `router.refresh()` to enter the feed and refresh the server-rendered route data.

Registration signs the user in immediately. There is no email verification, password-reset page, or social login in this version.

> **How to explain it**
>
> "Login and registration look related because they share one component. A single setting decides which fields and endpoint it uses. This keeps form behavior consistent and reduces repeated code."

04 / MAIN PAGE

The feed

Route: `/home` / File: `app/home/page.js`

The feed is the main workspace. It loads active posts through `useResource("/api/posts")` and renders each item with `PostCard`.

Element	Behavior
Page heading	"The feed" identifies the current view. Supporting copy introduces the tone of the community.
Post composer	A textarea stores the current draft in React state. Newlines are preserved in the displayed post.
Character counter	Shows the draft length out of 300. The server also enforces the 300-character limit.
Drop a thought	Submits trimmed text. Disabled for an empty draft, while posting, or while the feed is loading. A successful response is added to the displayed list and the draft is cleared.
Latest / Most liked	Latest follows the server's newest-first order. Most liked sorts a copy of the loaded posts by upvote count in the browser.
Refresh icon	Requests the feed again. Spins during loading and is disabled during loading, posting, or voting.
Post header	Shows a location icon, "Someone in [city]," the creation date/time, and a lifetime label.
Post text	Opens the full conversation when clicked.
Heart icon	Toggles a like through the vote API. The displayed count is updated from the server response.
Comment icon	Shows the comment count and links to the conversation.
Loading, empty, error states	Distinguish an unfinished request, a feed with no active posts, and a failed request.

State worth explaining

`content` is the draft, `posting` tracks submission, `voting` tracks a vote request, and `sort` selects the order. The resource hook owns the posts, loading state, and error message. No separate global state library is needed for these page-level interactions.

> **Be precise during the presentation**
>
> The feed shows all cities by default. The API accepts an optional `?city=Skopje` filter, but the current frontend does not expose a city filter or use geolocation. Visitors can read posts; posting and liking require authentication, which the server checks.

> **How to explain it**
>
> "The feed combines reading and creating in one view. Sorting happens locally, while posts and likes are saved on the server. The counters change after a successful response, so they reflect confirmed database operations."

05 / CONVERSATION PAGE

Post details and comments

Route: `/post/[id]` / File: `app/post/[id]/page.js`

This page focuses on one post. `useParams()` reads its ID from the URL, and `useResource()` requests the corresponding post, comments, and delete permission.

Element	Behavior
Back to the feed	Returns to the list without requiring the browser's Back button.
Detailed post	Reuses `PostCard` with `detail` enabled. Its body is text rather than another link to the same conversation.
Like button	Uses the same vote endpoint as the feed and updates the returned vote totals.
Delete icon	Visible only when the response contains `canDelete: true`. A browser confirmation appears before deletion.
Comments heading	Displays the current number of comments.
Add your voice	A textarea and 300-character counter collect a comment. The button requires nonblank text and an active post.
Comment list	Displays "Someone nearby," comment text, and creation time, with newest comments first.
Expired state	Shows an expired post label and disables new comments. Existing comments remain readable.

How the actions work

The local `act(action)` helper handles the repeated busy-state and error logic for liking, commenting, and deleting. On a successful comment, the returned comment is inserted into the list, the count increases, and the draft clears.

Deletion is checked again on the server. The server compares the signed-in user with the stored post owner, then removes the post, its comments, and its votes. The interface returns to the feed.

What "24 hours" actually means

The server sets `expiresAt` to the creation time plus 24 hours. Feed queries exclude expired posts. Expired records remain in MongoDB and can still appear on the owner's profile or be opened by their URL. The expiration index is a normal index, not a database TTL deletion rule.

The "24h drop" label is not a countdown. There is no live expiration timer on an already-open page. A new request recalculates expiration; the comment API independently rejects comments after expiry. Voting currently remains possible on expired posts.

> **How to explain it**
>
> "A post leaves the active feed after 24 hours, but its record is retained. Commenting is restricted by the server, while ownership determines who is allowed to delete the conversation."

Profile and About

My profile / `/profile`

File: `app/profile/page.js`

The profile presents the current account and its posts. It makes two separate requests: `/api/auth/me` for account information and `/api/posts/mine` for that user's posts.

Element	Purpose
Initial avatar	Uses the first letter of the username as a simple visual identifier.
Username and email	Identify the signed-in account. The profile endpoint excludes the password hash.
City	Shows the stored account city used when creating posts.
My drops and total	Show the owner's posts and their number, including expired posts.
Post list	Reuses `PostCard`. Likes are displayed as a count here; open the conversation to like or delete a post.
Loading and empty states	Explain whether data is still loading or the account has not posted anything yet.

If either protected request returns `401`, the resource hook redirects to login. Ownership filtering happens in the database query, using the authenticated user's ID.

The current profile is read-only. Editing the username, city, email, or password is not implemented.

> **How to explain the profile**
> "This page shows the relationship between a user and their posts. The server reads the user ID from the authentication token and uses it to query only that user's records."

About EchoDrop / `/about`

File: `app/about/page.js`

The About page contains a heading, the city photograph, the shared brand, and short text describing the project's purpose. It reuses `AppShell` to keep navigation consistent with the other main pages.

Its informational content does not require a database query. The page itself has no form or content mutation, although the shared navigation still includes the logout action.

> **How to explain About**
> "The About page gives the project context and uses the same navigation and visual identity as the rest of the application."

Components and visual design

File / component	Responsibility
`app/layout.js` RootLayout	Wraps all routes in the HTML document, sets language to English, imports global CSS, and defines the page title/description. It opts pages into dynamic rendering.
`components/brand.js` Brand	Combines the audio-lines icon, EchoDrop name, and accent dot in a link. A prop controls its destination.
`components/app-shell.js` AppShell	Provides the sidebar, active navigation, city image, logout control, top status text, content area, and footer. The page content is passed as `children`.
`components/auth-form.js` AuthForm	Provides the shared login/registration interface and submission logic.
`components/post-card.js` PostCard	Displays a post consistently across the feed, profile, and details page. Props decide whether its body links elsewhere and which actions appear.
`app/globals.css`	Defines colors, type, spacing, layout, forms, post styles, responsive breakpoints, and focus states.

Navigation and status

`usePathname()` identifies the active page so its navigation link can be highlighted. Icons support the text labels. The logo returns to the feed in the main layout. Logout clears the authentication cookie, then returns to login.

The green "Here for 24 hours" label describes the post lifetime. It is decorative status text, not a database-health check, an online-user counter, or a real-time connection indicator.

Design choices

The visual system uses a light neutral background, white content surfaces, coral primary actions, and green supporting accents. Narrow reading columns help scan posts; consistent spacing and small corner radii keep the interface quiet and organized. The photograph is a static asset, not a live image of the user's selected city.

On desktop, navigation sits in a sticky left sidebar. At 700px and below, it moves above the content, and the sidebar photograph is hidden. The authentication photograph becomes a short horizontal image. Additional adjustments at 1000px and 370px refine spacing for narrower screens.

Accessibility details

Forms have labels, icon buttons have accessible names and tooltips, keyboard focus is visible, and selected navigation and sorting controls expose their state. Errors use `role="alert"`; loading messages use `role="status"`. Reduced-motion preferences disable animation. These are implemented features, not a claim of a complete accessibility audit.

> **How to explain it**
>
> "I extracted elements that repeat across pages. A change to the post component or navigation now updates every place that uses it, making the interface easier to maintain."

MongoDB and the four models

Files: `lib/db.js` and `models/`

MongoDB stores documents in collections. Mongoose defines the expected fields and provides methods such as `find()`, `findById()`, and `create()`. All four schemas enable timestamps, adding `createdAt` and `updatedAt`.

Model	Main fields and rules
`User`	`username`: required, unique, trimmed, 3-50 characters. `email`: required, unique, lowercase, trimmed. `passwordHash`: required hashed password. `city`: required and trimmed.
`Post`	`content`: required, trimmed, maximum 300 characters. `city`: required location label. `user`: required reference to its owner. `expiresAt`: required indexed date. `upvoteCount` / `downvoteCount`: default to zero.
`Comment`	`content`: required, trimmed, maximum 300 characters. `post`: reference to the conversation. `user`: reference to the comment author.
`Vote`	`post` and `user`: required references. `isUpvote`: required Boolean. A unique compound index on post + user allows one vote record per user per post when the index exists.

Relationships

```
User --> many Posts
User --> many Comments <-- Post
User --> many Votes    <-- Post
```

References store MongoDB `ObjectId` values. `populate("user", "username")` loads the referenced username when preparing post responses. Comment counts are calculated with an aggregation that groups comments by post, avoiding a separate count query for every card.

Connection and configuration

`connectDb()` reads `MONGODB_URI` and calls `mongoose.connect()` with `dbName: "echodrop"`, explicitly selecting this project's database. It rejects a missing URI and uses a five-second server-selection timeout. Mongoose manages its normal connection pool; there is no custom global connection/promise cache.

`.env.local` contains the local connection URI and JWT secret and is excluded from git. `.env.example` provides placeholders. `npm run db:check` loads the environment, connects, pings MongoDB, and disconnects without changing records or printing credentials.

> **How to explain it**
>
> "The models separate the main entities. A comment refers to both a user and a post, and a vote does the same. That lets the application calculate counts and enforce ownership without copying an entire user into every document."

How authentication works

Files: `lib/auth.js` and `app/api/auth/`

Registering an account

1. The endpoint trims username and city, lowercases email, and checks required values.
2. It checks for an existing username or email.
3. `bcrypt.hash(password, 10)` produces a password hash, which is saved as `passwordHash`.
4. The server creates a signed JWT and sets an authentication cookie, immediately signing in the account.

Logging in

The server looks up the username and uses `bcrypt.compare()` to check the supplied password against the stored hash. It does not decrypt a stored password. Invalid credentials return a generic error; valid credentials produce a new token and cookie.

Token and cookie

Setting	Current implementation
JWT payload	User ID, username, and city. It is signed with `JWT_SECRET` and expires after seven days. Signing protects integrity; it does not make the payload encrypted.
Cookie name	`echodrop_token`, available for the application's root path.
`httpOnly`	Prevents frontend JavaScript from reading the cookie directly.
`secure`	Enabled in production, restricting cookie transmission to HTTPS.
Cookie lifetime	Seven days, matching the token's intended lifetime.
`sameSite`	The explicit `sameSite: "lax"` option is currently commented out. Do not present it as an explicitly configured application guarantee.

Recognizing the current user

`getCurrentUser()` reads the cookie, verifies the token, connects to MongoDB, and loads selected user fields. Requests without a usable login cannot perform protected actions. Logout deletes the cookie; it does not delete the account or implement a server-side token revocation list.

Authentication versus authorization

Authentication asks, "Who is making the request?" **Authorization** asks, "May this user perform this action?" A signed-in user can create a post, but only the post's owner can delete it. Hiding the delete button improves the interface; the API ownership check is the actual enforcement.

> **How to explain it**
>
> "Passwords are hashed in the database. A signed token in an HTTP-only cookie identifies the current account. The server still checks permission for actions such as deleting a post."

The API endpoints

Routes under `app/api/` exchange JSON with the frontend. GET reads data, POST creates data or performs an action, and DELETE removes a resource. In this table, `:id` means the actual post ID.

Method and route	Access / input	Result
`POST /api/auth/register`	Public. Username, email, city, password.	Creates the account, sets the cookie, and returns basic user information.
`POST /api/auth/login`	Public. Username and password.	Checks credentials, sets the cookie, and returns basic user information.
`POST /api/auth/logout`	No login required. No body needed.	Deletes the cookie and returns `{ ok: true }`.
`GET /api/auth/me`	Login required.	Returns ID, username, email, and city.
`GET /api/posts`	Public. Optional `city` query parameter.	Returns unexpired posts, newest first, with comment and vote counts.
`POST /api/posts`	Login required. `{ content }`.	Creates a post with the user's city and a 24-hour expiration date.
`GET /api/posts/mine`	Login required.	Returns the current user's posts, including expired ones.
`GET /api/posts/:id`	Public; uses login if present.	Returns the post, comments, and a `canDelete` flag.
`DELETE /api/posts/:id`	Login and ownership required.	Deletes the post and associated comments and votes.
`POST /api/posts/:id/vote`	Login required. `{ isUpvote: true }` for a like.	Creates, removes, or switches a vote and returns updated totals.
`POST /api/comments`	Login required. `{ postId, content }`.	Creates a comment if the post exists and has not expired.
`GET /api/comments/post/:postId`	Public.	Returns a post's comments, newest first. The detail page already receives comments from its post endpoint.

Status codes to recognize

`200` indicates a successful read or action; `201` indicates creation; `400` is used for rejected input; `401` means a login is missing or invalid; `403` means the user lacks permission; `404` means the target post is not found or its ID is invalid in the checked routes.

The vote model and endpoint also support downvotes, but the current interface only exposes likes. Clicking the same vote again removes it; switching direction updates the existing vote record and both counters.

> **How to explain it**
>
> "Each endpoint has a small, specific job. The frontend sends only the required input, and the server decides whether the operation is valid and permitted."

Shared logic and caching decisions

Helper	What it centralizes
`lib/api.js` `api()`	Sends requests with `cache: "no-store"`, converts request bodies to JSON, parses responses, and throws an error with the HTTP status when the response is unsuccessful. If the response lacks a usable JSON error, it uses a generic message.
`lib/use-resource.js` `useResource()`	Loads a URL, owns data/loading/error state, exposes setters and `reload()`, redirects on a 401 response, and aborts the request when the effect is cleaned up.
`lib/posts.js` `listPosts()`	Queries posts, loads referenced usernames, counts comments in one aggregation, and returns summaries. Used by the public feed and personal post list.
`lib/serialize.js`	`postSummary()` and `commentDto()` create plain response objects, convert IDs to strings, and select the fields sent to the browser. Post summaries calculate `isExpired`.
`lib/db.js` / `lib/auth.js`	Centralize database connection and authentication operations, so routes share one implementation.

What was simplified

Repeated login/registration forms became `AuthForm`. Repeated navigation became `AppShell`. Post rendering became `PostCard`. Request handling and list queries moved into small helpers. Pages now focus mainly on their own state, actions, and composition.

What "caching removed" means here

- The old custom global Mongoose connection/promise cache was removed.
- The root layout and GET API routes use `dynamic = "force-dynamic"`.
- The frontend request helper explicitly sets `cache: "no-store"`.
- `next.config.mjs` adds `Cache-Control: no-store, max-age=0` to API responses.
- Application links set `prefetch={false}`, avoiding automatic link prefetching.

Mongoose's connection pool and model registry still exist. React state still holds what is currently on screen. Framework build output and ordinary static assets are separate concerns. The change removes custom connection caching and application page/API data caching; it does not eliminate every kind of storage inside the browser, framework, or database.

There is also no polling or WebSocket feed. Other people's new posts appear on a later fetch, such as when Refresh is clicked. Removing caching does not itself create real-time updates.

> **How to explain it**
>
> "I reduced repeated code and made data requests explicit. A new feed request reads the database, while React state holds the current display. The database driver still reuses connections efficiently."

Follow one post from input to storage

Use this example when asked how the frontend and backend connect. Assume an authenticated user whose city is Skopje writes *"A good day for a walk through the city."*

1. **The draft changes.** The textarea's `onChange` updates `content`. React renders the new counter.
2. **The form submits.** `submitPost()` prevents navigation, trims the text, sets the busy flag, and calls `api()`.
3. **The browser sends JSON.** The request targets `POST /api/posts`. The same-origin authentication cookie accompanies it.
4. **The server identifies the user.** `getCurrentUser()` verifies the token and loads the account from MongoDB.
5. **The server validates the content.** Empty content and content exceeding 300 characters are rejected.
6. **The server sets trusted fields.** The user ID and city come from the authenticated account, not from arbitrary author fields supplied by the browser. Expiration is set on the server.
7. **Mongoose saves the document.** `Post.create()` stores the content, owner reference, city, and expiration date.
8. **The server returns a summary.** The response has status 201 and includes the new ID, timestamps, counts, and other summary fields.
9. **The frontend updates.** The returned post is inserted into React state, the draft clears, and the busy flag resets. A page reload can retrieve the same record from MongoDB.

The request body

```
{
  "content": "A good day for a walk through the city."
}
```

The request does not need an author ID, city, expiration date, or initial vote count. Those values are assigned through the authenticated account, route logic, and model defaults.

Two small JavaScript patterns

`setPosts(current => [post, ...current])` expresses "put the new post before the existing list." The actual page also handles an initially empty/null list. A functional state update uses the most recent state available to React.

`current.map(...)` updates a single post's vote counts by ID. Spreading the old post into a new object preserves its other fields. This is an immutable update: it creates the next state instead of modifying the old object in place.

> **How to explain it in one breath**
>
> "The browser sends the message text. The API authenticates the user, validates the message, and adds the owner, city, and expiration. MongoDB saves it, and the returned record is then added to the feed."

A practical 10-minute demo

Time	Show	Explain
0:00-1:00	Login screen and project introduction.	Purpose, audience, and React + Next.js + MongoDB architecture.
1:00-2:00	Create an account or log in.	Shared form, validation, hashed passwords, and authentication cookie.
2:00-4:00	Create a short post, like it, switch sorting, and refresh.	Component state, API requests, server-confirmed counts, and persistent storage.
4:00-5:30	Open the post, add a comment, and reload.	Dynamic route ID, post/comment relationship, and saved data surviving reload.
5:30-6:30	Open My profile and About.	Ownership filtering, shared components, and the informational page.
6:30-8:00	Show selected code files.	Trace `submitPost()` to its API handler, model, and database helper.
8:00-9:00	Narrow the browser; then delete the demo post.	Responsive navigation and server-side deletion permission.
9:00-10:00	Log out and discuss improvements.	Session cookie removal, current limits, and next development steps.

Prepare before presenting

Use a demonstration account and a recognizable test post. Open the app, verify MongoDB, and start the server if needed. Use the URL printed by Next.js; the previous session used `http://127.0.0.1:3000`.

Keep `app/home/page.js`, `app/api/posts/route.js`, `models/Post.js`, `lib/db.js`, and `components/post-card.js` ready. Explain one complete request before opening more files.

Useful commands

```
npm run dev       # Start the development server
npm run db:check  # Verify MongoDB connectivity
npm run lint      # Run ESLint checks
npm run build     # Create and verify a production build
npm start         # Run the production build
```

What has been checked

In the preceding implementation session, lint, build, and MongoDB checks passed. Browser checks covered account flows, password visibility, posting, liking, API cache headers, persisted comments, profile, and deletion. Desktop and 390px/320px layouts were checked; the completed workflow had no browser page errors.

These are development checks. The temporary browser script and screenshots are in ignored TestResults/. Playwright was installed locally and is not a declared dependency. There is no npm test command or committed regression suite.

> **Closing statement you can use**
>
> "EchoDrop demonstrates the complete journey from an interactive form to a validated API request and a persisted database record. Shared components keep the frontend consistent, and the backend controls identity, ownership, and the post lifecycle."

Accurate answers and next steps

Is EchoDrop fully anonymous?

No. The interface hides author names, but MongoDB retains user references, and `postSummary()` currently includes `username` in public post responses. Describe it as an anonymous-style interface. Stronger anonymity would require changing the response fields and making explicit privacy decisions.

Are expired posts automatically deleted?

No. Expiration removes them from the active feed and blocks new comments on the server. Their records remain readable through the profile and direct post URLs. There is no TTL deletion rule or scheduled cleanup task.

What makes this a full-stack project?

It includes the interactive frontend, HTTP API handlers, authentication and business rules, and a persistent MongoDB data layer in the same repository.

Why use Mongoose if MongoDB already stores documents?

Mongoose adds the model definitions, validation rules, references, and query methods used by the application. MongoDB is the database itself.

What does "use client" mean?

It marks the client-component boundary for interactive code that uses state, effects, event handlers, or browser APIs. Next.js can still produce initial HTML for client components; it does not mean the whole application is browser-only.

Does disabling caching make the feed real-time?

No. Each new request reads current data, but the interface must make that request. There is no automatic polling or WebSocket subscription.

Can a user delete someone else's post by calling the API?

The delete endpoint checks authentication and compares the stored owner ID with the current user. A different user receives a 403 response even if they bypass the interface.

Reasonable future improvements

These are development opportunities, not completed features:

- Remove author names from public responses if stronger anonymity is a requirement.
- Add pagination, a visible city filter, and a clearer signed-out state for protected actions.
- Add profile editing, password recovery, and moderation/reporting.
- Standardize server-side request validation and error responses across all routes.
- Make multi-document vote updates and deletion more robust under concurrent requests or partial failure. The current operations are not wrapped in a transaction.
- Add explicit cookie policy, request-abuse protection, and a repeatable automated test suite before a broader deployment.

Five terms to remember

Component: a reusable piece of interface. **State:** data that changes what React displays. **Endpoint:** a URL and HTTP method handled by the server. **Schema:** a definition of document fields and rules. **DTO/serialization:** selecting and shaping the data returned to the client.

Source of this guide: the current EchoDrop pages, components, API routes, helpers, schemas, configuration, README, and the preceding development verification results. Recheck behavior after future code changes.